

Plan d'améliorations pour l'application de recherche PubMed

Suppression de la limite de 200 articles (recherche illimitée)

Actuellement, la recherche est plafonnée à 200 résultats, ce qui empêche d'explorer l'ensemble des articles disponibles. Pour permettre une recherche sur un nombre **illimité** d'articles :

- **Adapter le paramètre `retmax` de l'API PubMed** : La requête ESearch utilise par défaut `retmax=20` dans le code actuel, ce qui limite à 20 PMIDs récupérés même si `max_results=200` ¹. Il faut retirer cette limite de test et définir `retmax` à une valeur élevée (par exemple 100000) ou calculée dynamiquement en fonction du besoin. D'après la documentation NCBI, une requête ESearch peut retourner jusqu'à **100 000** identifiants en une fois en ajustant `retmax` ². Au-delà, on peut effectuer plusieurs appels en paginant avec `retstart`. L'objectif est de ne plus tronquer la recherche à 200 articles, mais de récupérer la **totalité des résultats** correspondant aux critères.
- **Mettre en place la pagination côté backend si nécessaire** : Si le nombre de résultats dépasse la limite supportée par une requête (100k IDs par appel), implémentez une boucle pour appeler ESearch en plusieurs fois (en utilisant `retstart` pour décaler les résultats) afin de couvrir tous les articles. Assurez-vous également que la pagination côté frontend (paramètres `page` et `per_page`) fonctionne toujours en se basant sur la liste complète des résultats (`all_articles`).
- **Gestion des performances** : La récupération de milliers d'articles peut être coûteuse en temps et en mémoire. Il peut être judicieux d'avertir l'utilisateur ou de limiter l'intervalle de dates (`period`) par défaut. Cependant, le système devrait techniquement être capable de récupérer autant d'articles que nécessaire en plusieurs requêtes, sans imposer arbitrairement une limite de 200.

Utiliser les résultats réels de PubMed au lieu des données simulées

Dans la version actuelle, le backend bascule systématiquement sur des données simulées même lorsque PubMed répond correctement. En production, il faut **désactiver ce fallback automatique** pour afficher les vrais articles :

- **Modifier la fonction de recherche avec fallback** : Dans `fetch_real_pubmed_data_with_fallback`, une fois que la connexion à PubMed a réussi et que des PMIDs sont obtenus (`id_list` non vide), le code devrait enchaîner sur la récupération des détails des articles **réels**. Actuellement, la fonction se contente de faire un `break` puis retourne des données simulées (« données de démonstration ») quel que soit le cas ³. On voit même un commentaire indiquant que ce comportement est temporaire, et un `return []` sert

de placeholder après la collecte des IDs ⁴. Il faut remplacer ce comportement par une récupération effective des articles.

- **Implémenter la récupération des détails (EFetch)** : Après ESearch (obtention de la liste des PMIDs), utilisez l'utilitaire EFetch de l'API NCBI pour obtenir les **données complètes** de chaque article (titre, auteurs, résumé, etc.). En effet, ESearch ne fournit que les identifiants ⁵. Il est possible d'appeler EFetch via une requête HTTP (ex: `efetch.fcgi?db=pubmed&id=...&retmode=xml`) avec la liste de PMIDs. Vous pouvez récupérer jusqu'à 10 000 articles par appel EFetch ⁶, mais pour fiabiliser, on peut envoyer les requêtes par lots (par exemple 200 à 500 PMIDs par requête) afin d'éviter les erreurs serveur ⁷ ⁸.
- **Parser et utiliser les données réelles** : Le code comporte déjà une fonction `process_pubmed_xml` conçue pour extraire les informations d'un XML PubMed ⁹ ¹⁰. Servez-vous-en pour construire la liste d'articles à partir du résultat d'EFetch. Les champs tels que titre, auteurs, journal, année, DOI, résumé etc. seront ainsi remplis avec les vraies données de PubMed (avec un fallback "non disponible" si un champ manque). Assurez-vous de bien retourner cette liste réelle d'articles. En production, **le fallback simulé ne doit être utilisé qu'en cas d'échec** de la requête réelle (p. ex. si PubMed ne répond pas ou retourne zéro résultat). Ainsi, l'application affichera par défaut les vrais articles issus de PubMed, et n'utilisera les données factices qu'en dernier recours.
- **Nettoyer les éléments de test** : Supprimez ou désactivez les impressions de débogage et les limites de temps trop basses utilisées en phase de test (ex: `timeout=5s`). En production, on peut définir un timeout raisonnable (quelques secondes) et éventuellement réessayer via l'objet session avec stratégie de `Retry`. L'import `disable_warnings(InsecureRequestWarning)` pourra être nettoyé également. Ces ajustements garantiront que le backend cherche d'abord les **données réelles** et offre une expérience fidèle à l'utilisateur.

Améliorer l'analyse automatique des résumés d'articles

L'analyse basique actuelle (extraction de la taille d'échantillon, du critère de jugement principal, etc.) repose sur des fonctions utilitaires (`extract_sample_size`, `extract_primary_outcome`, etc.) dont les heuristiques peuvent manquer de précision si le résumé est complexe. Pour fiabiliser et enrichir cette analyse :

- **Examiner et affiner les méthodes existantes** : Vérifiez l'implémentation des fonctions d'extraction dans `app.utils`. Il est probable qu'elles utilisent des expressions régulières ou des mots-clés pour repérer, par exemple, un nombre d'échantillon ou un critère principal. Vous pourriez améliorer ces patterns en prenant en compte plus de variantes de langage. Par exemple, pour la **taille d'échantillon**, détecter des expressions comme *"randomized (n=120 patients)"*, *"we enrolled 120 participants"*, *"sample size of 120"*, etc. Pour le **critère de jugement principal**, chercher des formulations type *"primary outcome was X"*, *"main endpoint: X"*, ou repérer la présence de phrases concluant sur un résultat principal. Multipliez les cas de test avec des abstracts réels pour affiner ces fonctions.
- **Utiliser des techniques de NLP si possible** : Si la complexité des résumés dépasse ce que des regex peuvent capturer, envisagez d'utiliser des bibliothèques de Traitement du Langage Naturel. Par exemple, **spaCy** ou **NLTK** peuvent aider à détecter des entités numériques ou médicales. Il existe aussi des modèles entraînés sur les abstracts médicaux qui pourraient extraire automatiquement des éléments PICO (Population, Intervention, Comparison, Outcome).

Cela représente un travail conséquent, mais peut grandement améliorer la précision. À défaut de ML avancé, une approche intermédiaire est d'utiliser des listes de mots-clés médicaux (par ex. *patients, participants, outcome, endpoint, mortality, survival, etc.*) pour guider l'extraction.

- **Vérification manuelle et score de confiance** : Considérez calculer un score de confiance ou un indicateur de validation pour ces extractions. Par exemple, la variable `validation_score` est calculée comme le pourcentage d'articles avec données complètes ¹¹. On pourrait étendre cela en signalant si l'extraction du critère principal ou de l'échantillon semble incertaine (par ex., si plusieurs nombres sont présents dans le résumé ou si aucun mot clé clair n'est trouvé, marquer la valeur comme 'Non identifié'). L'objectif est de garantir que les données présentées (taille d'échantillon, critère) sont fiables, ou sinon de notifier qu'aucune information claire n'a pu être extraite.
- **Évolutivité de l'analyse** : Préparez le code à accueillir de futurs raffinements (par exemple, extraction d'autres informations comme le design de l'étude, la durée du suivi, etc.). En organisant le code d'analyse de façon modulaire, on pourra brancher de nouvelles fonctions d'extraction ou même un modèle de NLP entraîné ultérieurement, sans refondre tout le backend.

Connexion réelle à PubMed pour les résumés et métadonnées

Pour enrichir les informations affichées, il est essentiel de **connecter réellement l'application à PubMed** pour obtenir les résumés complets et les métadonnées des articles :

- **Inclure le résumé complet** : Grâce à l'appel EFetch mentionné plus haut, on récupérera le champ `AbstractText` de chaque article. Assurez-vous de bien remplir le champ `article['abstract']` (et éventuellement un résumé abrégé `summary`). Le code montre déjà comment concaténer les différentes sections de résumé si nécessaire ¹². Ces résumés complets permettront à l'utilisateur d'avoir directement l'information principale sans devoir aller sur PubMed pour chaque article.
- **Récupérer les métadonnées clés** : L'appel EFetch (en XML ou JSON) fournit de nombreuses données utiles : la liste complète des **auteurs**, le **journal** et la référence bibliographique, l'**année** de publication, le **DOI**, le **PMID**, le type d'étude (via les tags de publication type, ou en inférant par la présence de termes comme "Randomized Controlled Trial" dans les metadata). Le parser XML dans `process_pubmed_xml` extrait déjà la plupart de ces champs (par ex. Titre, Auteurs, Journal, Année, DOI) ¹³ ¹⁴. Il faudra peut-être compléter pour d'autres champs si nécessaire. Par exemple, vérifier si le type d'étude peut être directement extrait des tags MeSH ou du champ `PublicationType` (sinon, continuer d'utiliser `determine_study_type` sur le résumé comme actuellement).
- **Vérifier la cohérence des données** : Une fois les données réelles intégrées, testez sur plusieurs exemples pour voir si tout se remplit correctement. Par exemple, certains articles peuvent ne pas avoir de DOI, ou l'année de publication peut se trouver dans `MedlineDate` au lieu de `Year` (le code gère déjà ce cas en extrayant l'année d'une date type "2023 Jan" ¹⁵). Continuez dans cette lancée pour gérer les cas particuliers (articles très anciens sans année structurée, auteurs manquants, etc.), afin de rendre l'application robuste.
- **Désactivation du mode simulation en production** : Finalement, une fois la connexion PubMed fonctionnelle, le mode simulation ne devrait s'activer qu'en cas de **véritable besoin**. En production, on souhaite toujours afficher de vrais articles. Vous pouvez néanmoins garder une

option de secours (par exemple, si l'API NCBI atteint la limite de débit ou est hors service, éventuellement informer l'utilisateur et/ou charger une petite sélection d'articles prédéfinis plutôt que de ne rien afficher). Mais cette logique de secours doit être bien délimitée et documentée, pour ne pas interférer avec les résultats réels.

Raffinement des analyses statistiques et visualisations

Actuellement, l'onglet d'analyse fournit quelques statistiques de base (nombre d'articles, pourcentage avec résumés, etc.) ¹⁶. Il est possible d'aller plus loin pour offrir à l'utilisateur une meilleure compréhension des résultats :

- **Analyse temporelle des publications** : Utilisez les données par année déjà collectées (`analysis['years']`) pour afficher l'évolution du nombre de publications au fil du temps. Par exemple, générez un **graphique de tendance** montrant combien d'articles de la requête ont été publiés chaque année sur la période choisie. PubMed fournit déjà ce genre de graph sur son interface web (distribution des résultats par année). Vous pouvez reproduire cela dans votre application en front-end. Par exemple, intégrez une bibliothèque de graphes JavaScript (comme **Chart.js** ou Google Charts) pour tracer un histogramme des publications par année à partir des données de `analysis['years']`. Cela permettra de visualiser si le sujet est en plein essor récent ou s'il était plus étudié à une certaine époque.
- **Répartition par type d'étude ou journal** : De même, avec `analysis['study_types']` et `analysis['journals']`, on peut construire des **diagrammes en secteurs (pie chart)** ou des **barres** indiquant la proportion de chaque type d'étude dans les résultats, ou les journaux les plus représentés. Par exemple, un camembert montrant que 60% des articles trouvés sont des essais cliniques, 30% des revues systématiques, etc., peut être parlant pour l'utilisateur. Attention toutefois à ne pas surcharger l'interface : peut-être offrir ces visualisations dans un onglet ou une section "Analyses détaillées".
- **Tendances plus avancées** : Si pertinent pour l'utilisateur, on pourrait calculer la **variation du nombre de publications** sur la période (croissance ou déclin en pourcentage). Ou encore, coupler les données : par exemple montrer l'évolution du nombre d'essais cliniques vs revues systématiques sur les X dernières années dans le domaine recherché. Ces ajouts demandent de raffiner la fonction `analyze_trends` ou d'en créer de nouvelles. Assurez-vous que les calculs sont justes et testez-les avec des requêtes connues (ex: un sujet général comme "cancer" devrait montrer une augmentation constante du nombre de publications par an ¹⁷).
- **Intégration des graphiques dans le rapport** : Si vous exportez un PDF, pensez à inclure ces visualisations (par exemple, générer une image PNG du graphe et l'insérer dans le PDF). Pour l'interface web, assurez-vous que le JSON d'analyse renvoyé par `/api/analysis` contient les données nécessaires pour construire les graphiques côté client. L'ajout de ces visuels rendra l'application plus interactive et informative.

Export des résultats au format Excel avec colonnes spécifiques

L'export actuel se fait en CSV avec de nombreuses colonnes (titre, auteurs, journal, année, PMID, DOI, etc.) ¹⁸ ¹⁹. La demande est d'obtenir un fichier **Excel** ne contenant que certaines colonnes : **Titre**, **Critère de jugement**, **Échantillon**, **Journal** et **Année de publication**. Pour répondre à ce besoin :

- **Réduire et réorganiser les colonnes dans l'export** : Modifiez la génération du CSV pour n'inclure que les 5 champs souhaités, dans l'ordre spécifié. Concrètement, au lieu de :

```
writer.writerow([title, authors, journal, year, pmid, doi, study_type,
sample_size, primary_outcome, keywords, summary, url])
```

il faudra écrire par exemple :

```
writer.writerow([title, primary_outcome, sample_size, journal, year])
```

en veillant aussi à ajuster la ligne d'en-têtes correspondante. Dans la version française, on pourrait utiliser "**Titre**", "**Critère de jugement**", "**Échantillon**", "**Journal**", "**Année**" comme en-têtes. Si nécessaire, corrigez l'intitulé "Critère principal" actuel en "Critère de jugement" pour correspondre à la terminologie demandée (ce champ correspond au critère de jugement principal/outcome principal de l'étude). N'oubliez pas de conserver le bon encodage (UTF-8) pour les caractères spéciaux.

- **Format Excel (.xlsx) vs CSV** : Si un vrai fichier Excel est requis (extension `.xlsx`), vous pouvez utiliser une bibliothèque Python comme **openpyxl** ou **pandas** pour générer un tableau Excel formaté. Toutefois, si un CSV avec extension `.csv` convient (Excel pouvant l'ouvrir directement), la modification ci-dessus suffit. Vous pouvez tout de même changer le `download_name` en `.xlsx` et le `mimetype` en `application/vnd.openxmlformats-officedocument.spreadsheetml.sheet` si vous générez un binaire Excel. Sinon, conserver un CSV est simple et universel. L'important est que le contenu du tableau respecte les colonnes demandées.
- **Vérifications** : Testez l'export sur quelques recherches pour vous assurer que les colonnes correspondent exactement aux attentes et que les données figurent bien dans chaque colonne. Par exemple, la colonne Échantillon doit contenir un nombre (taille d'échantillon) ou être vide/null si inconnu, la colonne Critère de jugement doit contenir une phrase résumant l'outcome principal s'il a été identifié, etc. Assurez-vous aussi que l'ordre des colonnes est correct. Une fois validé, ce fichier pourra être directement transmis ou ouvert dans Excel par l'utilisateur.

En suivant ces recommandations, l'application de recherche PubMed gagnera en **puissance** (recherche non limitée), en **fidélité** (données réelles au lieu de simulées), en **pertinence** de l'analyse (extractions et stats plus précises), et en **utilisabilité** (export adapté et visualisations claires). Chaque modification devra être testée pour garantir l'absence de régression, mais elles offriront à l'utilisateur final une expérience nettement améliorée et professionnelle.

2 5 6 The 9 E-utilities and Associated Parameters - The Insider's Guide to Accessing NLM Data - National Library of Medicine

<https://www.nlm.nih.gov/dataguide/eutilities/utilities.html>

7 8 Entrez Direct Release Notes - Entrez Programming Utilities Help - NCBI Bookshelf

<https://www.ncbi.nlm.nih.gov/books/NBK564895/>

17 About PubMed by Year

<https://esperr.github.io/pubmed-by-year/about.html>